

Рекомендации по развитию инструкций проекта Proteus / eCharts

На основе практики разработки и доработки сложных OKR-таблиц и интерактивных виджетов

Цель документа. Зафиксировать повторяющиеся проблемы и архитектурные решения, которые уже проявились в реальной работе, чтобы следующие доработки были быстрее, стабильнее и с меньшим числом регрессий.

1. Что уже хорошо закрыто текущими инструкциями

- Каркас Proteus: overlay, скрывание canvas, глобальный option, 7 блоков, отсутствие внешних зависимостей.
- Изоляция CSS через namespace и защита от утечки стилей в интерфейс Proteus.
- Состояние через window.__pvtState и корректное поведение при перерисовках.
- Тултипы в body, position: fixed, отдельный renderTip() без полного render() на hover.
- Механическая валидация синтаксиса и ряда стоп-багов через validate.py.
- SELF_CHECK для содержательной проверки соответствия макету и данным.

Главный пробел сейчас не в базовом каркасе, а в production-поведении сложных таблиц: поиск, сортировка, пагинация, копирование, перестановка колонок, sticky-блоки и производительность могут по отдельности работать правильно, но ломать друг друга после очередной правки.

2. Что добавить в PATTERNS.md

PATTERNS.md лучше оставить единым реестром реальных падений. Ниже — новые RETRO, которые имеет смысл добавить по итогам последних итераций.

RETRO	Симптом	Причина	Правило
23	Пагинация ломает поиск	Поиск выполняется по отрэндеренному DOM или текущей странице.	Поиск и фильтрация всегда по полной модели; pagination применяется после search/filter/sort.
24	Пагинация ломает копирование	Сору читает DOM или pageRows.	Сору/export строится из полного набора отфильтрованных строк, а не из DOM и не только из текущей страницы.
25	Пустые значения поднимаются при DESC	null / "" / NaN участвуют в обычном comparator.	Пустые значения всегда отправлять в конец независимо от направления сортировки.
26	UI лагает на 1000+ строках	В DOM одновременно находятся все строки и много обработчиков.	Рендерить только текущую страницу; поиск/сортировку делать над JS-моделью.
27	Tooltip уезжает за viewport	Позиция рассчитывается только вправо от target.	Измерять tooltip и выбирать сторону; ограничивать

			left/right/top/bottom по viewport.
28	Sticky-колонка не закрепляется	Sticky находится не в том scroll-container или не имеет left/z-index/background.	Горизонтальный скролл должен принадлежать таблице; sticky cells получают position:sticky, left, z-index и фон.
29	После соседней правки ломается clipboard	Export связан с текущей DOM-разметкой.	Единый buildExportRows() из модели + current column order; DOM не источник экспорта.
30	Перестановка колонок расходится с данными	Порядок хранится только в DOM.	state.columnOrder — единый источник для header, rows и export.
31	Подсветка поиска ломает HTML	Highlight делается replace по готовой HTML-строке.	Искать по raw normalized value; вывод сначала экранировать, потом безопасно подсвечивать текст.
32	Только системные поля считаются выбранными метриками	Проверяется просто selected.length > 0.	Дата/unit/другие lockedFields не считаются пользовательским выбором метрик.
33	Обязательное поле можно выключить	Все поля используют один toggle.	lockedFields нельзя deselect; контрол disabled или не показывается.
34	Группа и дочерние поля визуально разъезжаются	Разные padding/indent rules после collapse.	Категория и children используют единую grid/indent-систему.
35	Метрика без динамики показывает пустой блок динамики	UI строится только по наличию значения.	Использовать capability metadata, например hasTrend:false, и вообще не рендерить лишний блок.
36	Скрытие строк не решает performance	Все строки всё равно создаются в DOM.	Не создавать невидимые строки: сначала page slice, потом HTML.

3. Что добавить в SKILL.md

3.1. Жёсткий pipeline для сложных таблиц

Для таблиц с search/filter/sort/pagination/export нельзя использовать DOM как источник данных или состояния.

```
rawData → MODEL → filter/search → sort →  VISIBLE_ROWS
                                     export/copy: все      pagination →  PAGE_ROWS → DOM
```

Ключевое следствие: пагинация — только способ ограничить DOM. Она не должна менять семантику поиска, сортировки и копирования.

3.2. Состояние интерактива — только в state

- state.page / state.pageSize

- state.search
- state.sortKey / state.sortDir
- state.columnOrder
- state.collapsedGroups
- state.selectedFields

Любая новая функция сначала должна быть встроена в модель состояния, и только потом в DOM. Например, drag колонок меняет state.columnOrder, после чего UI перерисовывается из состояния.

3.3. Новая визуализация и точечная правка — разные режимы работы

Текущее требование «план до кода и дождаться подтверждения» полезно для новой визуализации, но избыточно для однозначной точечной правки существующего рабочего файла.

Предлагаемое правило: для новой визуализации — план до реализации; для точечной правки существующей — сразу выполнить минимальный diff, если задача однозначна.

4. Что добавить в SELF_CHECK.md

Добавить отдельный раздел I. Интерактив / regression checks:

- I1. Search работает по всему набору, а не только по текущей странице.
- I2. Pagination применяется после filter/search/sort.
- I3. Сору/экспорт включает все найденные строки, независимо от текущей страницы.
- I4. Сору/экспорт соблюдает текущий порядок колонок.
- I5. Пустые значения при ASC и DESC всегда внизу.
- I6. Смена search/filter/sort сбрасывает или нормализует текущую страницу.
- I7. После column reorder сортировка, поиск и export продолжают работать.
- I8. Locked/system fields нельзя выключить.
- I9. Если выбраны только locked/system fields, показывается empty-state для отсутствующих метрик.
- I10. Collapse группы не меняет выравнивание её элементов.
- I11. Tooltip не выходит за левую/правую границу viewport.
- I12. Tooltip не вызывает полный render().
- I13. Sticky-колонки проверены при горизонтальном и вертикальном scroll.
- I14. При rows > pageSize DOM содержит только примерно pageSize строк данных.
- I15. Highlight поиска не ломает HTML и спецсимволы.
- I16. Метрики с hasTrend=false вообще не рендерят область динамики.
- I17. После добавления новой функции повторно проверены search + sort + pagination + сору.

Особенно важен I17: несколько регрессий с копированием возникали не при разработке сору, а после добавления соседних функций. Нужен обязательный smoke-test связки интерактивов.

5. Что добавить в validate.py

В валидатор стоит добавлять только проверки, которые можно формально проверить без большого числа ложных срабатываний.

- **V1.** Clipboard/экспорт не должен строиться обходом строк DOM. Если рядом с clipboard/writeText используются querySelectorAll("tr"), innerText/table.inner... — WARN/FAIL.
- **V2.** Если есть pagination/pageRows, а сору использует pageRows — WARN: экспорт может копировать только текущую страницу.
- **V3.** При наличии сортировки числовых полей — WARN, если comparator не содержит явной обработки null/empty/NaN.

- **V4.** Если присутствует pagination, но buildHTML очевидно итерирует полную MODEL.rows — WARN о возможном лишнем DOM.

Не стоит переносить весь regression-checklist в validate.py: сила текущего валидатора в том, что PASS у него означает формально проверяемое свойство. Слишком много эвристик превратят его в шум.

6. Добавить новый файл REGRESSION.md

SELF_CHECK отвечает на вопрос «код соответствует контракту?». REGRESSION.md должен отвечать на другой вопрос: «мы не сломали уже работающие функции после изменения?»

Dataset

- ☐ 0 rows
- ☐ 1 row
- ☐ 99 rows
- ☐ 100 rows
- ☐ 101 rows
- ☐ 1000+ rows

Search

- ☐ match на первой странице
- ☐ match на дальней странице
- ☐ нет совпадений
- ☐ спецсимволы
- ☐ highlight

Sort

- ☐ ASC
- ☐ DESC
- ☐ null / empty
- ☐ после search
- ☐ после column reorder

Pagination

- ☐ first
- ☐ previous
- ☐ next
- ☐ last
- ☐ page recalculated after search

Copy

- ☐ без search — все строки
- ☐ с search — все matching rows
- ☐ не только текущая страница
- ☐ текущий порядок колонок
- ☐ скрытые/невывбранные колонки исключены
- ☐ корректная вставка в Excel

Columns

- ☐ select
- ☐ collapse group
- ☐ reorder
- ☐ locked date/unit

Tooltip

- ☐ центр
- ☐ правый край
- ☐ левый край
- ☐ нижний край

Scroll

- ☐ vertical
- ☐ horizontal
- ☐ sticky left block

7. Расширить рецепт производительности

В PATTERNS.md стоит расширить раздел про большие объёмы и зафиксировать практический default:

- До ~100 строк: обычный полный render обычно допустим.
- 100–1000 строк: для табличных интерфейсов предпочтительна пагинация.
- 1000+ строк: DOM обязательно ограничивать текущей страницей; search/sort выполнять над массивом данных.
- Частые события: debounce для поиска; никогда не делать full render на mousemove.
- Listeners: предпочтительно event delegation от overlay.
- Copy/export: строить текст напрямую из данных, без обхода DOM.

Границы не должны считаться строгими benchmark-порогами; это безопасные практические defaults.

8. Capability-driven UI для неоднородных метрик

Если разные метрики поддерживают разный набор функций, лучше описывать это metadata, а не накапливать условия по ключам метрик.

```
metrics: {
  fact: { hasTrend: true, sortable: true },
  expectedCompletion: { hasTrend: false, sortable: true }
}
```

Потенциальные capability-флаги:

- hasTrend
- sortable
- copyable
- searchable
- draggable
- locked
- defaultVisible
- format

9. Приоритет внедрения

Приоритет	Файл	Что сделать
1	PATTERNS.md	Добавить RETRO 23–36: это сразу зафиксирует реальные причины уже встречавшихся багов.
2	SKILL.md	Закрепить pipeline MODEL → filtered → sorted → export/page → DOM и правило state-first.
3	SELF_CHECK.md	Добавить раздел интерактива и обязательный regression smoke-test.
4	REGRESSION.md	Создать отдельный сценарный чек-лист для сложных таблиц.
5	validate.py	Добавить только несколько безопасных эвристик как WARN/FAIL.

10. Итог

Базовый контракт проекта уже хорошо защищает от инфраструктурных ошибок Proteus. Следующий уровень зрелости — зафиксировать архитектуру интерактивных таблиц и регрессионные сценарии. Самое важное правило: DOM — это только представление текущей страницы, а поиск, сортировка, порядок колонок и export должны жить на уровне модели и состояния.